UNITÀ
02
LEZIONE
01

Le liste in Python

Creazione, indici e primi metodi

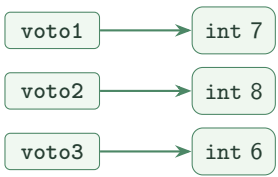
Prof. Francesco Gobbi
Anno scolastico 2026/2027
I.I.S.S. Galileo Galilei – Ostiglia (MN)

Che cos'è una lista

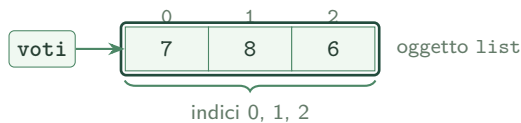
Una **lista** è un oggetto che raccoglie **più valori in sequenza**, sotto un'unica etichetta. Ogni valore occupa una **posizione**, indicata da un numero intero chiamato **indice**.

Esempio: i voti di uno studente. Senza lista servono tante etichette quanti sono i voti; con la lista basta un'etichetta sola.

Senza lista: tre etichette



Con la lista: un'etichetta sola



Con la lista, il numero di valori può **cambiare durante l'esecuzione**: si possono aggiungere o togliere elementi senza creare nuove etichette.

Le caratteristiche di una lista

- **Ordinata:** gli elementi mantengono l'ordine in cui vengono inseriti.
- **Indicizzata:** il primo elemento ha indice **0**, il secondo indice 1 e così via.
- **Modificabile (mutabile):** dopo la creazione, gli elementi possono essere aggiunti, tolti o sostituiti. L'oggetto lista resta **lo stesso**.
- **Eterogenea:** può contenere oggetti di tipo diverso, per esempio `str`, `int` e `bool` (slide seguente).
- **Di lunghezza variabile:** `len()` restituisce il numero di elementi presenti in quel momento.

`list` è una **classe**: ogni lista creata nel programma è un **oggetto** (un'istanza) di questa classe. Gli `int` e le `str` sono **immutabili**, la lista invece è **mutabile**.

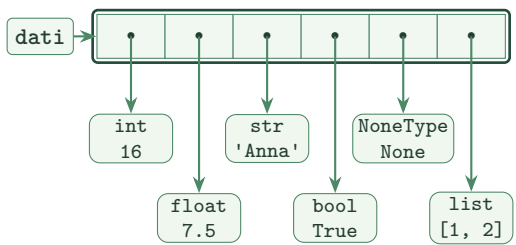
Quali oggetti può contenere una lista

Una lista può contenere oggetti di **qualsiasi classe**: `int`, `float`, `str`, `bool`, `None` e perfino **altre liste**. Gli elementi possono anche essere **di tipo diverso tra loro** nella stessa lista.

```
dati = [16, 7.5, "Anna", True, None, [1, 2]]
for elemento in dati:
    print(elemento, type(elemento))
```

OUTPUT

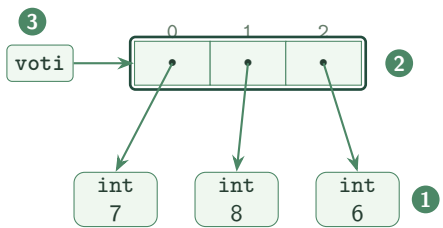
```
16 <class 'int'>
7.5 <class 'float'>
Anna <class 'str'>
True <class 'bool'>
None <class 'NoneType'>
[1, 2] <class 'list'>
```



La lista **non ha un tipo per gli elementi**: ogni cella indica un oggetto con la **propria classe**. Di solito conviene usare elementi **dello stesso tipo**, richiesti da `sum()` e `sort()`.

Come Python crea una lista in memoria

Per eseguire `voti = [7, 8, 6]` Python svolge tre passaggi:



oggetti `int`

1. Vengono valutati i letterali 7, 8, 6: si ottengono tre **oggetti** `int`.
2. Viene creato un **oggetto** `list` con tre celle. Ogni cella contiene un **riferimento** a un oggetto, non il valore stesso.
3. L'etichetta `voti` viene associata all'oggetto lista.

Una lista contiene **riferimenti a oggetti**. Gli schemi mostrano i riferimenti, non contano le allocazioni reali: per i piccoli interi CPython può riutilizzare oggetti già esistenti.

I modi per creare una lista

Una lista si crea scrivendo gli elementi tra **parentesi quadre**, separati da virgole, oppure con la funzione `list()`, che costruisce una lista a partire da un'altra sequenza.

```
vuota = []                      # Lista vuota
voti = [7, 8, 6]                # Tre interi
misti = ["Anna", 16, True]      # Tipi diversi
numeri = list(range(1, 6))      # Da un range
lettere = list("ciao")          # Da una stringa

print(vuota, voti, misti)
print(numeri, lettere)
print(type(voti), len(voti))
```

`[]` crea una lista **senza elementi**: è il punto di partenza quando gli elementi vengono aggiunti in seguito.

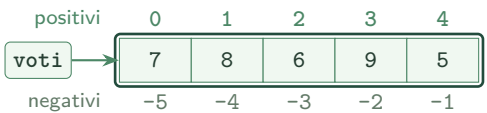
`list(range(1, 6))` trasforma l'intervallo in una lista di interi.

`type()` conferma la classe `list`; `len()` restituisce il numero di elementi.

```
OUTPUT  [] [7, 8, 6] ['Anna', 16, True]
        [1, 2, 3, 4, 5] ['c', 'i', 'a', 'o']
        <class 'list'> 3
```

Leggere un elemento con l'indice

`lista[i]` restituisce l'elemento in posizione `i`. Gli **indici positivi** partono da 0 a sinistra; gli **indici negativi** partono da -1 a destra.




```
voti = [7, 8, 6, 9, 5]
print(voti[0], voti[2])     # 7 6
print(voti[-1], voti[-2])   # 5 9
print(len(voti))            # 5
print(voti[1:3])            # [8, 6]
print(voti[5])              # IndexError
```

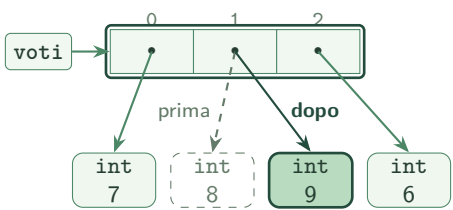
Per brevità, da qui in avanti il valore viene scritto **dentro la cella**: la cella contiene in realtà un riferimento all'oggetto.

L'ultimo indice valido è `len(voti) - 1`. `voti[1:3]` è una **sezione**: restituisce una **nuova lista** con gli elementi dagli indici 1 a 2 (il 3 è escluso, come in `range()`).

Cambiare un elemento della lista

L'assegnazione `voti[1] = 9` sostituisce il **riferimento** contenuto nella cella 1. L'oggetto lista resta **lo stesso**: il suo ID non cambia.

```
voti = [7, 8, 6]
print(id(voti))
voti[1] = 9       # Nuovo riferimento
print(voti)       # [7, 9, 6]
print(id(voti))   # Stesso ID di prima
```



L'intero 8 **non viene modificato** (gli `int` sono immutabili): la cella indica semplicemente un altro oggetto. La **lista**, invece, è stata modificata: è **mutabile**.

Le funzioni len(), sum(), max() e min()

Alcune funzioni già note ricevono una lista come argomento e restituiscono un valore. La lista **non viene modificata**.

```
voti = [7, 8, 6, 9]
print(len(voti))                # Quanti
print(sum(voti))                # Somma
print(max(voti), min(voti))     # Max e min
media = sum(voti) / len(voti)
print(media)
```

```
OUTPUT  4
        30
        9 6
        7.5
```

La **media** si calcola dividendo la somma per il numero di elementi: $30/4 = 7.5$.

`sum()` richiede elementi numerici; `max()` e `min()` richiedono elementi confrontabili.

Con una lista **vuota**, `len()` restituisce 0 e la media provocherebbe una divisione per zero: va controllato prima che la lista contenga elementi.

Percorrere una lista con il ciclo for

Il ciclo `for` può percorrere una lista in due modi: **per valore**, quando interessa solo l'elemento, oppure **per indice**, quando serve anche la posizione.

Per valore

```
voti = [7, 8, 6]
for voto in voti:
    print(voto)
```

```
OUTPUT  7
        8
        6
```

A ogni iterazione l'etichetta `voto` viene associata all'elemento successivo.

Per indice

```
voti = [7, 8, 6]
for i in range(len(voti)):
    print("Posizione", i, "voto", voti[i])
```

```
OUTPUT  Posizione 0 voto 7
        Posizione 1 voto 8
        Posizione 2 voto 6
```

`range(len(voti))` fornisce gli indici 0, 1, 2: proprio quelli validi.

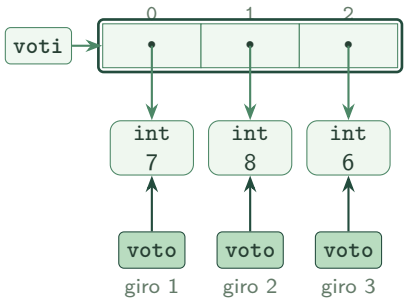
Il ciclo for e gli oggetti della lista

Nel ciclo `for voto in voti`, a ogni giro l'etichetta `voto` viene associata **allo stesso oggetto** indicato dalla cella: **non viene creata una copia**. Riassegnare `voto` cambia solo l'etichetta, **non la lista**.

```
voti = [7, 8, 6]
for voto in voti:
    voto = voto + 1       # Cambia solo voto
print(voti)               # Lista invariata

for i in range(len(voti)):
    voti[i] = voti[i] + 1    # Cambia la cella
print(voti)
```

```
OUTPUT   [7, 8, 6]
         [8, 9, 7]
```



Per **modificare** un elemento serve l'**indice**:
`voti[i] = Operazione/Valore`

Che cos'è un metodo

Un **metodo** è una funzione che appartiene a una classe. Si chiama **sull'oggetto**, con la **notazione puntata**: prima l'oggetto, poi il punto, poi il nome del metodo con gli argomenti.



	Funzione	**Metodo**
Esempio	`len(voti)`	`voti.append(9)`
Dove sta la lista	tra le parentesi	prima del punto
Effetto sulla lista	`len()` non la modifica	`append()` e `insert()` la modificano

In questa lezione vengono usati **`append()`** e **`insert()`**; gli altri metodi sono l'argomento della **lezione 02**.

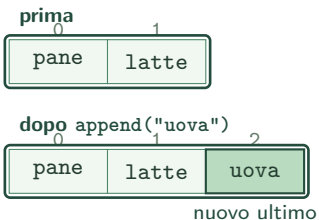
Il metodo append(): aggiungere in fondo

`append(x)` aggiunge l'oggetto `x` **in fondo** alla lista. La lista cresce di un elemento e resta **lo stesso oggetto**.

Sintassi: `lista.append(x)` **Modifica la lista:** sì **Restituisce:** `None`

```
spesa = ["pane", "latte"]
spesa.append("uova")      # In fondo
print(spesa)
print(len(spesa))
esito = spesa.append("riso")
print(esito)              # None
```

```
OUTPUT  ['pane', 'latte', 'uova']
        3
        None
```



Errore tipico: `spesa = spesa.append("uova")` associa `spesa` a `None` e la lista viene persa.

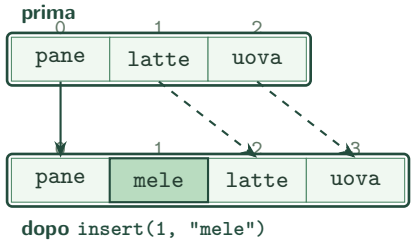
Il metodo insert(): inserire in una posizione

`insert(i, x)` inserisce `x` **nella posizione** `i`. Gli elementi che si trovano da `i` in poi **slittano di un posto verso destra**.

Sintassi: `lista.insert(i, x)` **Modifica la lista:** sì **Restituisce:** `None`

```
spesa = ["pane", "latte", "uova"]
spesa.insert(1, "mele")   # Posizione 1
print(spesa)
spesa.insert(0, "riso")   # In testa
print(spesa)
```

```
OUTPUT  ['pane', 'mele', 'latte',
        'uova']
        ['riso', 'pane', 'mele',
        'latte', 'uova']
```



`latte` e `uova` passano agli indici 2 e 3. Con un indice oltre la fine, `x` va in fondo.

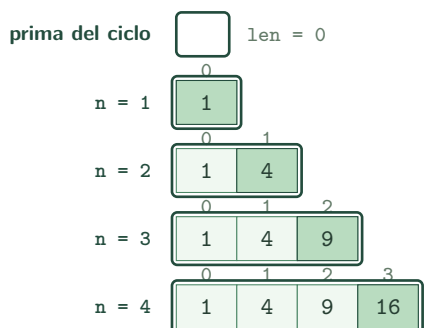
Riempire una lista con un ciclo

Spesso la lista parte **vuota** e viene riempita dentro un ciclo. Nell'esempio vengono raccolti i quadrati dei numeri da 1 a 4.

```
quadrati = []                # Vuota
for n in range(1, 5):
    quadrati.append(n * n) # In fondo
print(quadrati)
```

```
OUTPUT   [1, 4, 9, 16]
```

La lista è sempre **la stessa**: a ogni iterazione cresce di un elemento. `quadrati = []` va scritto **prima** del ciclo, altrimenti verrebbe svuotata a ogni giro.



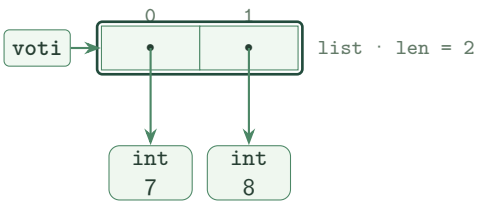
in verde intenso: l'elemento appena aggiunto

Seguire una lista riga per riga – riga 1 di 5

Codice: riga in esecuzione

```
1 voti = [7, 8]
2 voti.append(6)
3 voti.insert(0, 9)
4 voti[3] = 5
5 print(voti, len(voti))
```

Etichetta e oggetti dopo la riga 1



Vengono valutati i letterali 7 e 8 e viene creato un **oggetto `list`** con due riferimenti. L'etichetta `voti` viene associata alla lista.

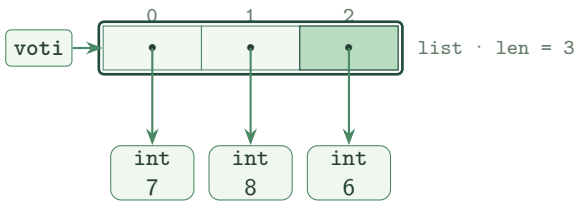
Riquadro con celle: oggetto `list` Freccia: riferimento Tratteggio grigio: nessun riferimento diretto

Seguire una lista riga per riga – riga 2 di 5

Codice: riga in esecuzione

```
1 voti = [7, 8]
2 voti.append(6)
3 voti.insert(0, 9)
4 voti[3] = 5
5 print(voti, len(voti))
```

Etichetta e oggetti dopo la riga 2



`append(6)` aggiunge **in fondo** un riferimento all'intero 6. L'oggetto lista è **lo stesso**: cambia solo il contenuto e `len` passa da 2 a 3.

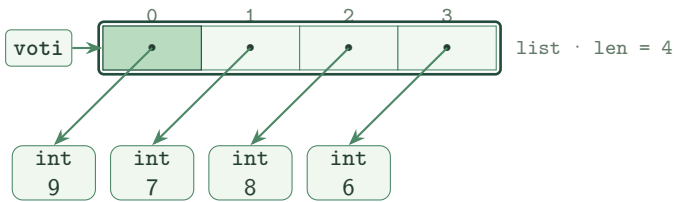
Riquadro con celle: oggetto `list` Freccia: riferimento Tratteggio grigio: nessun riferimento diretto

Seguire una lista riga per riga – riga 3 di 5

Codice: riga in esecuzione

```
1 voti = [7, 8]
2 voti.append(6)
3 voti.insert(0, 9)
4 voti[3] = 5
5 print(voti, len(voti))
```

Etichetta e oggetti dopo la riga 3



`insert(0, 9)` inserisce un riferimento in posizione 0. Gli oggetti **non si spostano**: **slittano i riferimenti**, e gli indici di 7, 8, 6 aumentano di 1.

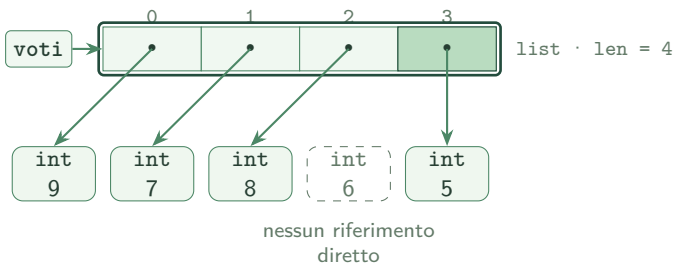
Riquadro con celle: oggetto `list` Freccia: riferimento Tratteggio grigio: nessun riferimento diretto

Seguire una lista riga per riga – righe 4 e 5 di 5

Codice: riga in esecuzione

```
1 voti = [7, 8]
2 voti.append(6)
3 voti.insert(0, 9)
4 voti[3] = 5
5 print(voti, len(voti))
```

Etichetta e oggetti dopo le righe 4 e 5



Riga 4: `voti[3] = 5` sostituisce il riferimento della cella 3; l'intero 6 **non ha più riferimenti diretti**. **Riga 5:** vengono mostrati la lista e la sua lunghezza.

```
OUTPUT  [9, 7, 8, 5] 4
```

Errori frequenti con le liste

- **Indice fuori dall'intervallo:** con 5 elementi gli indici validi vanno da 0 a 4 (l'ultimo è `len(voti) - 1`); `voti[5]` provoca `IndexError`.
- **Partire a contare da 1:** il primo elemento è `voti[0]`, non `voti[1]`.
- **Assegnare il risultato di `append()`:** `voti = voti.append(9)` associa `voti` a `None` e la lista viene persa.
- **Inizializzare la lista dentro il ciclo:** `lista = []` scritto nel corpo del ciclo svuota la lista a ogni giro.
- **Credere che `for voto in voti` modifichi la lista:** `voto = voto + 1` cambia solo l'etichetta; per cambiare la lista serve `voti[i]`.
- **Calcolare la media di una lista vuota:** `sum(voti) / len(voti)` con `len(voti) == 0` provoca `ZeroDivisionError`.

Esercizio 1: la media dei voti

Problema. Leggere `n` voti interi ($n \geq 1$) **memorizzandoli in una lista**. Mostrare la lista, la **media**, il voto **più alto**, il **più basso** e il numero di **sufficienze** (voti ≥ 6).

```
def main():
    n = int(input("Numero di voti: "))
    voti = []                       # Lista vuota
    for i in range(n):
        voto = int(input("Voto: "))
        voti.append(voto)           # In fondo
    media = sum(voti) / len(voti)
    sufficienze = 0                 # Contatore
    for voto in voti:               # Per valore
        if voto >= 6:
            sufficienze = sufficienze + 1
    print("Voti:", voti, "- Media:", media)
    print("Max:", max(voti), "- Min:", min(voti),
          "- Sufficienze:", sufficienze)
if __name__ == '__main__':
    main()
```

Caso di prova: $n = 4$, voti 7, 5, 8, 6.

OUTPUT

```
Voti: [7, 5, 8, 6] - Media: 6.5
Max: 8 - Min: 5 - Sufficienze: 3
```

Spiegazione

Riempimento: `voti = []` sta **prima** del ciclo; ogni `append()` aggiunge in fondo il voto letto.

Media: `sum(voti) / len(voti)`, cioè $26/4 = 6.5$.

Massimo e minimo: `max()` e `min()`, senza ciclo.

Sufficienze: il contatore aumenta per 7, 8 e 6.

Esercizi:

2. I numeri sopra la media
Creare un programma in Python che legga `n` numeri interi, li memorizzi in una lista con `append()`, calcoli la **media** e crei una **nuova lista** con i soli numeri **maggiori della media**. Mostrare la media e la nuova lista.
Prova: 4, 10, 6, 8 → media `7.0`, sopra la media `[10, 8]`.

3. Le temperature della settimana
Data `temperature = [18, 21, 17, 24, 22, 19, 25]`, **creare un programma in Python** che mostri **massima**, **minima** e **media**, poi crei una **nuova lista** con le **posizioni** (indici) dei giorni sopra la media.
Prova: massima 25, minima 17, media 20.857...; posizioni `[1, 3, 4, 6]`.

4. Comprensione del codice
Analizzare il codice **senza eseguirlo**: **commentare** ogni riga, **disegnare lo schema** etichetta – lista – oggetti `int` dopo ogni riga e **prevedere l'output**. Solo dopo, verificarlo eseguendo il codice.

```
1 a = [3, 6]
2 a.append(9)
3 a.insert(1, 4)
4 a[0] = a[-1]
5 print(a, len(a), sum(a))
```